

FRONTIERS IN COMPUTING: DISCUSSION 3

RENTO SAIJO

2026-09-15

Protocols and network services

- A network protocol defines the rules and messages that let devices communicate. Agreement about message contents and responses makes communication possible across different manufacturers' equipment. Address Resolution Protocol, or ARP, illustrates this cooperation: a host uses a request and response to discover the MAC address associated with an IPv4 address on its local network. RFC 826 specifies the exchange so different implementations can follow the same rules. This local address resolution helps deliver traffic to a neighboring device, including a router that carries it toward another network.
- Other protocols organize particular services. File Transfer Protocol (FTP) transfers files between clients and servers; its **RETR** command requests retrieval of a file. Simple Mail Transfer Protocol (SMTP) supports email transfer, including exchanges between mail servers. These protocols give participants a shared vocabulary of commands and responses. A server is a computer running software that understands and answers the relevant requests. Consequently, an FTP server, mail server, and web server offer different application services while relying on the same underlying principles of host-to-host communication.
- Hypertext Transfer Protocol (HTTP) governs exchanges of web resources. A browser can send a **GET** request, and a successful server response can include the status 200 OK followed by the requested content. Transport Layer Security (TLS) protects the exchange between browser and server; HTTP carried over this encrypted connection is HTTPS. SSL is the older predecessor to TLS. The protocol names therefore describe related responsibilities: HTTP structures the application conversation, while TLS protects that conversation as information travels between the communicating endpoints.

Addressing and automatic configuration

- The host-configuration example brings four IPv4 settings together. An IP address identifies the host's network connection, and a subnet mask helps determine which destinations are local. The prefix /24 and mask 255.255.255.0 express the same network size. A default gateway identifies the router used to reach destinations beyond the local network. Finally, a DNS server provides name resolution: the Domain Name System translates names such as website domains into IP addresses. DNS also helps locate mail servers. Naming and addressing work together so people can use recognizable names while hosts communicate using network addresses.
- Dynamic Host Configuration Protocol (DHCP) supplies configuration automatically. When a laptop joins a coffee shop's Wi-Fi, it can begin a DHCP exchange with a discovery message and obtain an IP address, subnet mask, default gateway, and DNS-server information. This explains why users can browse without manually entering those settings whenever they join a network. DHCP prepares the host to communicate, while DNS helps it locate named destinations afterward. Together with ARP and routing, these services connect an ordinary action, such as opening a website, to the addressing information needed to deliver traffic.

From a URL to a web page

- Entering a Uniform Resource Locator (URL) gives the browser instructions about what to retrieve. The scheme, such as `https`, identifies how to communicate; the domain identifies the named host; and the path identifies the requested resource. A path may resemble a directory and filename, helping us understand how one site offers many resources. Before contacting the server, the browser needs its address. It can use cached DNS information from the browser or operating system, or ask a DNS resolver to obtain an answer through the DNS infrastructure.
- Once the address is known, the TCP-based exchange shown in the browser example requires a connection with the server. Establishing that connection involves a handshake; HTTPS adds a TLS handshake to establish the protected exchange. These setup steps take time before the requested content arrives. Connection reuse allows later requests to use an existing connection, and TLS session resumption can reduce the work required when reconnecting. DNS caching, connection reuse, and session resumption improve loading by avoiding repeated work at different stages of the same communication process.
- The browser then sends an HTTP request, the server processes it, and an HTTP response carries the result back. Rendering the returned HTML often reveals additional resources to retrieve, including images and JavaScript files. Loading a page can therefore require several exchanges rather than a single response containing everything visible on screen. Cached name information and reusable connections can support these additional requests. We can trace the overall progression from a readable address, through name resolution and connection setup, to resource delivery and the browser's construction of the page.

HTML, CSS, and JavaScript

- Hypertext Markup Language (HTML) describes a page's content and structure. Markup identifies elements such as headings, paragraphs, and lists, allowing the browser to organize the document. Hyperlinks connect a resource to other resources, creating the linked documents of the World Wide Web. The newspaper analogy helps distinguish structured content from an undifferentiated collection of words: a headline and a paragraph serve different purposes. HTML provides this foundation for web pages, which are delivered through the Internet's communication infrastructure.
- Cascading Style Sheets (CSS) control presentation, including typography, colors, backgrounds, borders, spacing, and layout. The same underlying content can take different visual forms as its style rules change. CSS also supports responsive design, in which a layout adapts to the available screen space. The video contrasts viewing a page on a large television with viewing it on a smartphone: the content remains usable because its arrangement can change. Structure and styling therefore cooperate to make information readable across devices with very different displays.
- JavaScript adds programmable behavior, allowing a page to respond to events and update what users see. Clicking a shopping-cart control, for example, can trigger code that changes the displayed cart. HTML defines the relevant page elements, CSS determines their appearance, and JavaScript coordinates the response to the action. These technologies work together in the browser's front end. More complete interactions, such as storing a cart or signing into an account, also involve server-side processing and data, connecting the visible interface to the broader application behind it.

Working across the full stack

- Full-stack development combines work on the client-facing front end, the server-side back end, and the connections between them. The front end presents information and receives user actions. The back end applies business rules, handles data, and performs tasks such as authentication. Application Programming Interfaces (APIs) define ways for application components to request services and exchange information. Understanding both sides allows a developer to follow an interaction beyond the screen, through the server's processing, and back to the response presented to the user.
- Code Institute organizes this work around data as presented, data in motion, and data at rest. Data as presented is the information visible in a browser, mobile application, dashboard, or another interface. In this framework, data in motion refers to data participating in the processes and algorithms that determine application behavior. Data at rest is stored for later access. Front-end work concentrates on presentation; back-end work includes processing and storage. The distinction helps us see how an application transforms a user action into a result while preserving information needed beyond that interaction.
- The role extends across design, implementation, testing, debugging, maintenance, and collaboration with other developers. Frameworks and libraries provide reusable capabilities, while version control with tools such as Git supports coordinated changes. Working across the stack also involves databases, servers, and API integration. This breadth helps developers investigate problems that cross component boundaries and understand how their decisions affect the complete application. Continued learning matters because the tools evolve, and developers who later specialize still benefit from understanding the components their own work depends on.

Databases, infrastructure, and MERN

- Back-end technologies supply the processing and storage capabilities behind the interface. Languages such as Python, Java, and C# can implement application logic; Node.js provides a runtime for server-side JavaScript. Database systems manage stored application data. SQL supports storing, modifying, and retrieving information in systems such as PostgreSQL and MySQL, while the assignment also identifies NoSQL systems such as MongoDB. These are technology choices within the stack's responsibilities. A developer needs to connect the chosen interface, processing code, and database into a consistent flow of requests, results, and stored information.
- The assignment extends this picture to DevOps and infrastructure: hosting, deployment, scaling, and monitoring keep an application available and maintainable after development. Its examples include Docker, Kubernetes, GitHub Actions, and cloud platforms such as Azure, AWS, and Google Cloud. These tools support different operational tasks, from packaging software to running and updating it. An application therefore involves both code that implements its features and an environment that operates those features. Deployment and monitoring connect development decisions to the behavior users experience once the system is running.
- The MERN example combines MongoDB for data storage, Express for the back-end web framework, React for the user interface, and Node.js for the server-side runtime. JavaScript spans the client and server application code. Returning to the shopping-cart example, React can present the cart, an HTTP request can carry an action to Express running on Node.js, and server logic can store or retrieve the relevant data in MongoDB. The response then supports an updated interface. Following this flow brings networking protocols, browser behavior, application logic, and persistent data together in one functioning system.